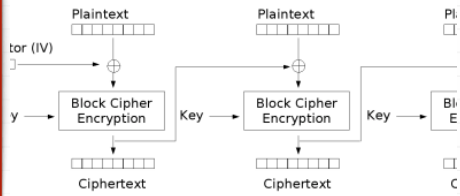
# Cybersecurity

## Modern ciphers – attacks




Cipher Block Chaining (CBC) mode encryption



## Agenda

- Attacks against stream ciphers
- Attacks block ciphers
  - DES,2DES,3DES strenght
  - Differential Cryptanalysis
  - Linear Cryptanalysis
- Padding Oracle Attack



## Berlekamp-Massey Algorithm

- Given (part of) a (periodic) sequence, can find shortest LFSR that could generate the sequence
- Berlekamp-Massey algorithm
  - Order $L^2$, where L is length of LFSR
  - Iterative algorithm
  - Only $2L$ consecutive bits required



## Berlekamp-Massey Algorithm

- Binary sequence: $s = (s_0, s_1, s_2, \ldots, s_{n-1})$
- **Linear complexity** of $s$ is the length of shortest LFSR that can generate $s$
- Let L be linear complexity of $s$
- Then connection polynomial of $s$ is of form
  $C(x) = c_0 + c_1x + c_2x^2 + \ldots + c_Lx^L$
- Berlekamp-Massey finds L and $C(x)$



## Berlekamp-Massey Algorithm

```
// Given binary sequence s = (s_0, s_1, s_2, ..., s_{n-1})
// Find linear complexity L and connection polynomial C(x)
BM(s)
    C(x) = B(x) = 1
    L = N = 0
    m = -1
    while N < n // n is length of input sequence
        d = s_N ⊕ c_1 s_{N-1} ⊕ c_2 s_{N-2} ⊕ ··· ⊕ c_L s_{N-L}
        if d == 1 then
            T(x) = C(x)
            C(x) = C(x) + B(x) x^{N-m}
            if L ≤ N/2 then
                L = N + 1 - L
                m = N
                B(x) = T(x)
            end if
        end if
        N = N + 1
    end while
    return(L)
end BM
```

sequence: $s = (s_0, s_1, \ldots, s_7) = 10011100$
initialize: $C(x) = B(x) = 1,\ L = N = 0,\ m = -1$

$N = 0$
$d = s_0 = 1$
$T(x) = 1,\ C(x) = 1 + x$
$L = 1,\ m = 0,\ B(x) = 1$

$N = 1$
$d = s_1 \oplus c_1 s_0 = 1$
$T(x) = 1 + x,\ C(x) = 1$

$N = 2$
$d = s_2 \oplus c_1 s_1 \oplus c_2 s_0 = 0$

$N = 3$
$d = s_3 \oplus c_1 s_2 \oplus c_2 s_1 \oplus c_3 s_0 = 1$
$T(x) = 1,\ C(x) = 1 + x^3$
$L = 3,\ m = 3,\ B(x) = 1$

$N = 4$
⋮



## Cryptographically Strong Sequences

- A sequence is **cryptographically strong** if it is a "good" keystream
  - "Good" relative to some specified criteria
- Crypto strong sequence must be **unpredictable**
  - Known plaintext exposes part of keystream
  - Trudy must not be able to determine more of the keystream from a short segment
- Small linear complexity implies predictable
  - Due to Berlekamp-Massey algorithm

Cybersecurity

## Cryptographically Strong Sequences

- Necessary for a **cryptographically strong** keystream to have a high linear complexity
- But not sufficient!
- Why? Consider $s = (s_0, s_1, \ldots, s_{n-1}) = 00\ldots01$
- Then $s$ has linear complexity $n$
  - Smallest shift register for $s$ requires $n$ stages
  - Largest possible for sequence of period $n$
  - But $s$ is not cryptographically strong
- Linear complexity "concentrated" in last bit

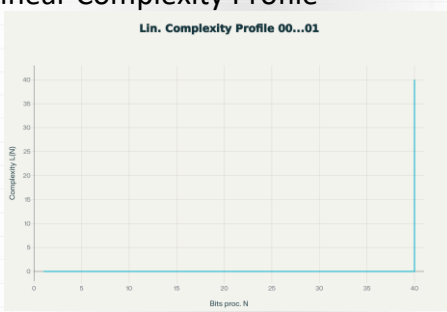
Cybersecurity

## Linear Complexity Profile

- **Linear complexity profile** is a better measure of cryptographic strength
- Plot linear complexity as function of bits processed in Berlekamp-Massey algorithm
  - Should follow $n/2$ line "closely but irregularly"
- Plot of sequence $s = (s_0, s_1, \ldots, s_{n-1}) = 00\ldots01$ would be 0 until last bit, then jumps to $n$
  - Does **not** follow $n/2$ line "closely but irregularly"
  - Not a strong sequence (by this definition)

Cybersecurity

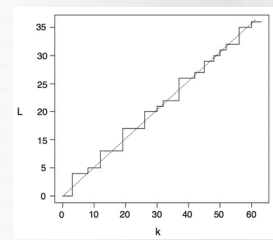
## Linear Complexity Profile

- 


Cybersecurity

## Linear Complexity Profile

- A "good" linear complexity profile




Cybersecurity

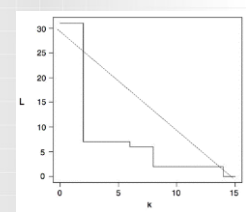
## k-error Linear Complexity Profile

- Alternative way to measure cryptographically strong sequences
- Consider again $s = (s_0, s_1, \ldots, s_{n-1}) = 00\ldots01$
- This $s$ has max linear complexity, but it is only 1 bit away from having min linear complexity
- k-error linear complexity is min complexity of any sequence that is "distance" $k$ from $s$
- 1-error linear complexity of $s = 00\ldots01$ is 0
  - Linear complexity of this sequence is "unstable"

Cybersecurity

## k-error Linear Complexity Profile

- k-error linear complexity profile
  - k-error linear complexity as function of k
- Example:
  - Not a strong $s$
  - Good profile should follow diagonal "closely"



Cybersecurity

## Crypto Strong Sequences

- Linear complexity must be “large”
- Linear complexity profile must $n/2$ line “closely but irregularly”
- $k$-error linear complexity profile must follow diagonal line “closely”
- All of this is necessary but not sufficient for crypto strength!

Cybersecurity

## Correlation Attack

- Trudy obtains some segment of keystream from LFSR stream cipher
  - of the type considered on previous slides
- Can assume stream cipher is the multiple shift register case
  - If not, convert it to this case
- By Kerckhoffs Principle, we assume shift registers and combining function known
- Only unknown is the key
  - The key consists of LFSR initial fills

Cybersecurity

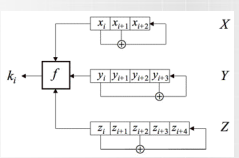
## Correlation Attack

- Trudy wants to recover LFSR initial fills
  - She knows all connection polynomials and nonlinear combining function
  - She also knows $N$ keystream bits, $k_0, k_1, \ldots, k_{N-1}$
- Sometimes possible to determine initial fills of the LFSRs independently
  - By correlating each LFSR output to keystream
  - A classic **divide and conquer** attack

Cybersecurity

## Correlation Attack

- For example, suppose keystream generator is of the form:




- And $f(x,y,z) = xy \oplus yz \oplus z$
- Note that key is 12 bits - initial fills

Cybersecurity

## Correlation Attack

- For stream cipher on previous slide
- Suppose initial fills are
  - $X = 011, Y = 0101, Z = 11100$

| | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $x_i$ | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| $y_i$ | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| $z_i$ | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| $k_i$ | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

Cybersecurity

## Correlation Attack

- Consider truth table for combining function: $f(x,y,z) = xy \oplus yz \oplus z$
- Easy to show that
  - $f(x,y,z) = x$ with probability $3/4$
  - $f(x,y,z) = z$ with probability $3/4$
- Trudy can use this to recover initial fills from known keystream

Cybersecurity

## Correlation Attack

- Trudy sees keystream in table
- Trudy wants to find initial fills
- She guesses $X = 111$, generates first 24 bits of putative $X$, compares to $k_i$

| $x_i$ | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $k_i$ | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

- Trudy finds 12 out of 24 matches
- As expected in random case

Cybersecurity

## Correlation Attack

- Now suppose Trudy guesses correct fill, $X = 011$
- First 24 bits of $X$ (and keystream)

| $x_i$ | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $k_i$ | 1 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |

- Trudy finds 21 out of 24 matches
- Expect 3/4 matches in causal case
- Trudy has found initial fill of $X$

Cybersecurity

## Correlation Attack

- How much work is this attack?
  - The $X,Y,Z$ fills are 3,4,5 bits, respectively
- We need to try about half of the initial fills before we find $X$
- Then we try about half of the fills for $Y$
- Then about half of $Z$ fills
- Work is $2^2 + 2^3 + 2^4 < 2^5$
- Exhaustive key search work is $2^{11}$

Cybersecurity

## Correlation Attack

- Work factor in general…
- Suppose $n$ LFSRs
  - Of lengths $N_0, N_1, \ldots, N_{n-1}$
- Correlation attack work is $2^{N_0-1} + 2^{N_1-1} + \cdots + 2^{N_{n-1}-1}$
- Work for exhaustive key search is $2^{N_0+N_1+\cdots+N_{n-1}-1}$

Cybersecurity

## Conclusions

- Keystreams must be **cryptographically strong**
  - Crucial property: unpredictable
- Lots of theory available for LFSRs
  - Berlekamp-Massey algorithm
  - Nice mathematical theory exists
- LFSRs can be used to make stream ciphers
  - LFSR-based stream ciphers must be **correlation immune**
  - Depends on properties of function $f$

Cybersecurity

## Strength of DES – Key Size

- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- Brute force search looks hard
- Recent advances have shown is possible
  - in 1997 on Internet in a few months
  - in 1998 on dedicated hardware (EFF) in a few days
  - in 1999 above combined in 22hrs!
  - In 2008 - the RIVYERA machine reduced the average time to less than one single day.

Cybersecurity

## Strength of DES – Timing Attacks

- Attacks actual implementation of cipher
- Use knowledge of consequences of implementation to derive knowledge of some/all subkey bits
- Specifically use fact that calculations can take varying times depending on the value of the inputs to it

Cybersecurity

## Strength of DES - Complementation Property

- DES is a Feistel network, thus:

$$DES(\neg m, \neg k) = \neg DES(m, k)$$

- This is called the "Complementation Property"
- So:

if $c_1 = DES(m, k)$ and $c_2 = DES(\neg m, k)$ then if

$DES(m, k_1) \neq c_1$ nor $\neg c_2$ then $k \neq k_1$ nor $\neg k_1$

- Search space is reduced by half

Cybersecurity

## Strength of 2DES

- Double encryption with DES (2DES) is bad

$$2DES_{k1k2}(m) = E_{k1}(E_{k2}(m))$$

- 2DES is vulnerable to "meet in the middle" attack
  - i.e. for a fixed message `m` create a table:

$$E_k(m) \text{ for all } k \in \{0,1\}^{56}$$

- Then for $c = E_{k1k2}(m)$ try all k until $D_{k1}(c)$ is in the table
- Thus 2DES can be broken on average in $2^{56}$ operations using $2^{56}$ memory slots (a time-space tradeoff)- not good for 112 bits of key (56 + 56)

Cybersecurity

## Strength of 3DES

- Two-key Triple DES (3DES): DES three times, two keys (112 bits)

$$3DES_{k1k2}(m) = E_{k1}(D_{k2}(E_{k1}(m)))$$

- The strength of DES (& 3DES) is that it does not form a group

$$DES_{k1}(DES_{k2}(m)) \neq DES_{k3}(m)$$

Cybersecurity

## Strength of DES – Analytic Attacks

- Now exist several analytic attacks on DES
- Utilise some deep structure of the cipher
  - by gathering information about encryptions
  - can eventually recover some/all of the sub-key bits
  - if necessary then exhaustively search for the rest
- Statistical attacks
  - differential cryptanalysis
  - linear cryptanalysis
  - related key attacks

Cybersecurity

## Differential Cryptanalysis

- One of the most significant public advances in cryptanalysis
- Known in 70's with DES design
- Murphy, Biham & Shamir published 1990
- Powerful method to analyse block ciphers
- Used to analyse most current block ciphers with varying degrees of success
- DES reasonably resistant to it

Cybersecurity

## Differential Cryptanalysis

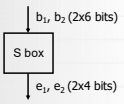
- A statistical attack against Feistel ciphers
- Design of S-P networks has output of function *f* influenced by both input & key
- Hence cannot trace values back through cipher without knowing values of the key
- Differential Cryptanalysis compares two related pairs of encryptions

Cybersecurity

## Differential Cryptanalysis

- We consider any s-box function $F(x, k_i)$ :
- Define the difference measure (on input) as

$$\Delta = b_1 \oplus b_2$$
$$= (x_1 \oplus k_i) \oplus (x_2 \oplus k_i)$$
$$= x_1 \oplus x_2$$




- The input XOR ($b_1 \oplus b_2$) does not depend on the key but the output XOR ($e_1 \oplus e_2$) does.

Cybersecurity

## Differential Cryptanalysis

- Some input difference giving some output difference with probability p
- If we find instances of some higher probability input / output difference pairs occurring we will be able to infer subkey that was used in a round
- Then must iterate process over many rounds

Cybersecurity

## Differential Cryptanalysis

- Perform attack by repeatedly encrypting plaintext pairs with known input XOR until obtain desired output XOR
- when found
  - if intermediate rounds match required XOR have a **right pair**
  - if not then have a **wrong pair**
- can then deduce keys values for the rounds
  - right pairs suggest same key bits
  - wrong pairs give random values
- Larger numbers of rounds makes it more difficult
- Attack on full DES requires an effort on the order of $2^{47}$, requiring $2^{47}$ chosen plaintexts to be encrypted

Cybersecurity

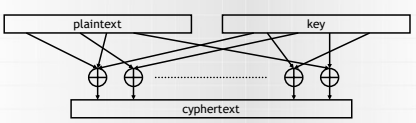
## Linear Cryptanalysis

- Also a statistical method
- Based on finding linear approximations to model the transformation of DES
- Can attack DES with $2^{47}$ known plaintexts,
  - still in practise infeasible

Cybersecurity

## Linear Cryptanalysis

- **Consider the cyphertext derived by combining certain bits from the plaintext and key:**




- **The cypher can easily be broken, for example, if**

$$c[1] = p[4] \oplus p[17] \oplus k[5] \oplus k[3]$$
$$\text{i.e. } k[3] \oplus k[5] = c[1] \oplus p[4] \oplus p[17]$$

- **This is because the cypher is linear.**

Cybersecurity

## Linear Cryptanalysis

- **If we use the following notation:**

$$p[i_1, \ldots, i_u] = p[i_1] \oplus p[i_2] \oplus \ldots \oplus p[i_u]$$
(the xor bits of the plaintext)

- **Also define**

$$q = Pr\ [\ p[i_1, \ldots, i_u] \oplus c[j_1, \ldots, j_v] = k\ [s_1, \ldots, s_w]\ ]$$

- **Now if $|q - 0.5|$ is large, then we can guess k $[s_1, \ldots, s_w]$**
- **Optimally for a break $|q - 0.5| = 0.5$ ($q = 0$ or 1)**
**(perfect cipher would have $q = 0.5$)**

Cybersecurity

## Linear Cryptanalysis

- **In 1993, Matsui made the following observation about DES which approximates the 5th S-box as a linear function:**

$$q_5 = Pr[\ x[4] = S_5(x)[0,1,2,3]\ ] = 12/64 = 0.19$$
(Note $x \in \{0,1\}^6$)

- **For the ith DES round**

$$Pr_i\ [r_i[15] \oplus F(r_i, k_i)[7,18,24,29] = k_i[22]] = p_5 = 0.19$$

**($r_i$ is the ith round right hand part)**

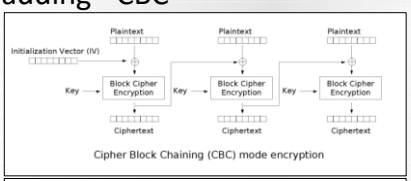
**where the bits have been chosen to undo the permutation.**

Cybersecurity

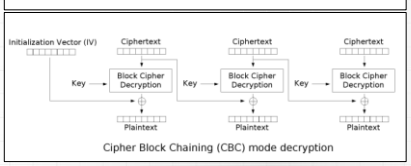
## DES strengths against attacks

| Attack | Complexity | | | |
|---|---|---|---|---|
| | # messages | | requirements | |
| | known | chosen | storage | processing |
| exhaustive precomputation | - | 1 | $2^{56}$ | 1 (lookup) |
| exhaustive search | 1 | - | neg. | $2^{55}$ |
| linear cryptanalysis | $2^{43}$ (85%) | - | for texts | $2^{43}$ |
| | $2^{38}$ (10%) | - | for texts | $2^{50}$ |
| differential cryptanalysis | - | $2^{47}$ | for texts | $2^{47}$ |
| | $2^{55}$ | - | for texts | $2^{55}$ |

## Padding - CBC



Cipher Block Chaining (CBC) mode encryption



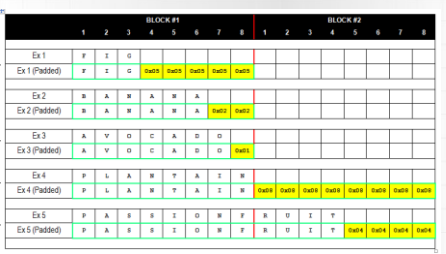
Cipher Block Chaining (CBC) mode decryption

## Background - Padding

- Why padding?
  - Plaintext messages come in a variety of lengths.
  - Block ciphers require all messages to come in an exact number of blocks.

Padding is added into the plaintext, not the ciphertext.

## Background - Padding



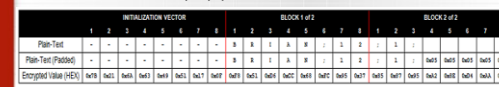

At least one padding byte is ALWAYS appended

## Background – Padding + Oracle

- The final decrypted block should end with:
  - » A single 0x01 byte (0x01)
  - » Two 0x02 bytes (0x02, 0x02)
  - » Three 0x03 bytes (0x03, 0x03, 0x03)
  - » Four 0x04 bytes (0x04, 0x04, 0x04, 0x04)
  - » ...and so on
- If not, most cryptographic providers will throw an invalid padding exception.
  - This extra information is called ***Oracle***.

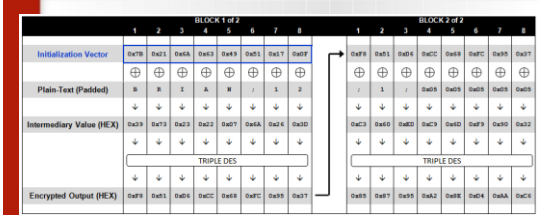
## A Basic Padding Oracle Attack Scenario

- An application uses a query string parameter
  - to pass the encrypted username, company id, and role id of a user
  - http://sampleapp/home.jsp?UID=7B216A634951170FF851D6CC68FC9537858795A28ED4AAC6
    - Ciphertext in ASCII Hex representation, 24bytes.
  - Plaintext: **BRIAN;12;2;**



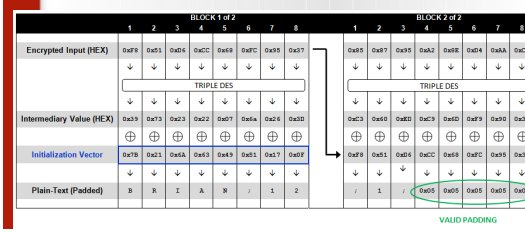

## Understand the whole process for the correct plaintext

- Encryption Diagram




## Understand the whole process for the correct plaintext

- Decryption Diagram



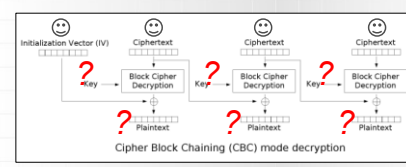

## The Padding Oracle in Web Apps

- When the application receives an encrypted value, it responds in one of three ways:
  - When a valid ciphertext is received (one that is properly padded and contains valid data) the application responds normally (200 OK).
  - When an invalid ciphertext is received (one that, after decrypted, does not end with a valid padding) the application throws a cryptographic exception (500 Internal Server Error, or 403...).
  - When a valid ciphertext is received (one that is properly padded) but decrypted to an invalid value, the application displays a custom error message (404 Not Found).

We can distinguish valid padding or not.

## Know our attack goal and resources

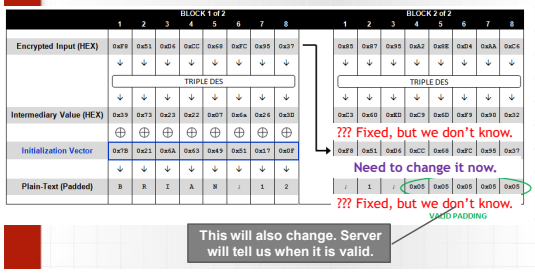
- Our goal: decrypt the value by using padding oracle attack.




- Moreover, we have the padding oracle information that server will respond.

## The Overview of the PO Attack

- The attack trick: isolate each block and try to only decrypt this block of plaintext.

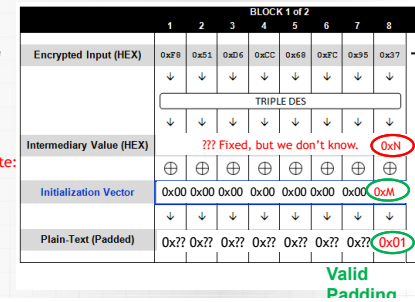



## The Overview of PO Attack

- If we can change them to this status:

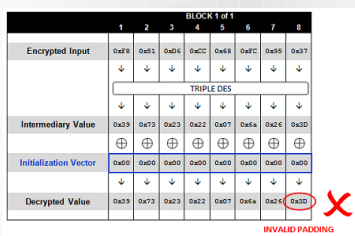
Get 0xN :
- 0xM: we try and know
- 0x01: server tells us

Get one byte:
- = 0xN XOR 0x3D



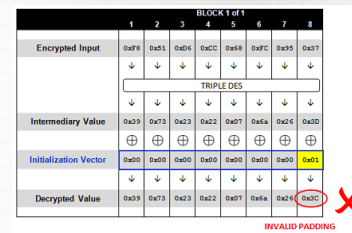

## Recap the detailed attack steps

- First try from 0 (an IV of all NULL values):
  - Request:http://sampleapp/home.jsp?UID=00000000000000F851D6CC68FC9537
  - Response: 500 - Internal Server Error



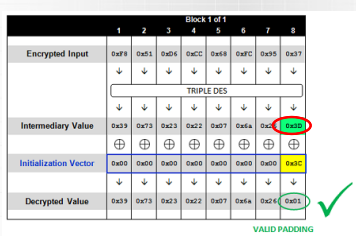

## Recap the detailed attack steps

- Second try is 1:
  - Request:http://sampleapp/home.jsp?UID=00000000000001F851D6CC68FC9537
  - Response: 500 - Internal Server Error

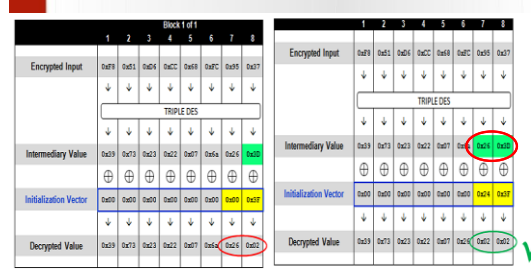



## Recap the detailed attack steps

- Until this try:
  - Request:http://sampleapp/home.jsp?UID=000000000000003CF851D6CC68FC9537
  - Response: 200 OK




## Recap the detailed attack steps



## Recap the detailed attack steps

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Encrypted Input | 0xF8 | 0x51 | 0xD6 | 0xCC | 0x68 | 0xFC | 0x95 | 0x37 |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| | TRIPLE DES | | | | | | | |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| Intermediary Value | 0x39 | 0x73 | 0x23 | 0x22 | 0x07 | 0x6a | 0x26 | 0x3D |
| | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ |
| Initialization Vector | 0x31 | 0x7B | 0x2B | 0x2A | 0x0F | 0x62 | 0x2E | 0x35 |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| Decrypted Value | 0x08 | 0x08 | 0x08 | 0x08 | 0x08 | 0x08 | 0x08 | 0x08 |

VALID PADDING

## Recap the detailed attack steps

- Finally recover the plaintext for the first block

| | BLOCK 1 of 2 | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| Encrypted Input (HEX) | 0xF8 | 0x51 | 0xD6 | 0xCC | 0x68 | 0xFC | 0x95 | 0x37 |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| | TRIPLE DES | | | | | | | |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| Intermediary Value (HEX) | 0x39 | 0x73 | 0x23 | 0x22 | 0x07 | 0x6a | 0x26 | 0x3B |
| | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ | ⊕ |
| Initialization Vector | 0x7B | 0x21 | 0x6A | 0x63 | 0x49 | 0x51 | 0x17 | 0x0F |
| | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ | ↓ |
| Plain-Text (Padded) | | | | | | | | |


**Cybersecurity**

Block Ciphers – Attacks